TRIBHUVAN UNIVERSITY

INSTITUE OF ENGINEERING
PULCHOWK CAMPUS

A LAB REPORT
on
Projects Using
Django

SUBMITTED BY:

Name: Subesh Yadav
Roll: 080BCT084
Group: D
Lab No: 4
Date: 3/5/2026

SUBMITTED TO:

Prakash Chandra Sir

Department of Electronics
and Computer Engineering

Theory

1. Handling Requests & Responses

Django is a **web framework** that works on the **request-response cycle**. Every time a user interacts with the web application, the browser sends an HTTP request to the server. Django processes this request via the appropriate view, which may interact with the database, apply business logic, or validate input, and then returns an HTTP response to the client. This separation allows developers to manage input, processing, and output efficiently while keeping the code organized and maintainable.

2. Form Data Handling & Sessions

Form Data Handling:
Django provides forms to securely process input data. Forms handle input validation, error messages, and clean data submission, ensuring that only correct data reaches the backend. They also simplify rendering fields in templates while providing consistent styling and validation feedback.

Sessions:
Sessions allow storing user-specific data across multiple requests. Django uses session IDs stored in cookies to track users. This enables functionality like maintaining login states, tracking activity, or storing temporary data securely on the server side.

3. Routing

Routing is the process of mapping URLs to the corresponding views in the application. Django's URL dispatcher interprets the incoming request URL and directs it to the appropriate view function or class. Parameters can be extracted directly from URLs, which allows dynamic handling of resources, such as editing or deleting a specific task by its unique identifier.

4. Middleware

Middleware in Django is a layer of hooks that runs **before and after a request** reaches a view. Middleware can modify requests, handle responses, log user activity, enforce security, and handle errors. It acts as a central place for cross-cutting concerns like logging, authentication, security headers, and error handling.

5. Templating Concepts

Django templates separate presentation from logic. Templates allow dynamic rendering of HTML by using placeholders and template tags. Features include:

- Loops and conditionals to render lists or conditional content
- Template inheritance to maintain consistent layouts across pages
- Filters and custom tags for formatting data

This ensures that HTML structure is consistent, maintainable, and easily adaptable to user-specific data.

6. Database Integration: Relational vs NoSQL, CRUD Operations, ORM Concept

Relational vs NoSQL:
Django typically uses relational databases like SQLite, MySQL, or PostgreSQL, where data is organized into tables with rows and columns. NoSQL databases, in contrast, store unstructured data and are suitable for flexible or large-scale datasets.

CRUD Operations:

- **Create:** Add new data entries to the database
- **Read:** Retrieve and query existing data
- **Update:** Modify existing entries
- **Delete:** Remove entries

ORM (Object-Relational Mapping):
Django's ORM allows developers to interact with the database using Python objects instead of SQL queries. This abstraction simplifies database operations, improves security, and ensures consistency between models and database schema.

7. Authentication & Authorization: Cookies, Sessions

Authentication verifies the identity of a user, while **authorization** controls access to resources based on permissions. Django provides an authentication system with secure password handling, login/logout mechanisms, and permission checks.

Cookies & Sessions:

- Cookies store session IDs in the user's browser.
- Sessions map the session ID to user-specific data stored server-side.
- This allows the application to maintain user state across multiple requests and enforce access control to resources such as personal tasks.

8. Middleware for Logging, Error Handling, and Security

Logging: Middleware can record every user request, including timestamps, user identity, session ID, and cookies, providing an audit trail for debugging and monitoring.

Error Handling: Middleware can catch exceptions and provide user-friendly error messages, preventing the application from crashing.

Security: Middleware also enforces security by protecting against CSRF attacks, checking authentication, and validating user permissions before granting access to sensitive pages.

Project Description

The **Student Task Manager** is a web-based application built using Django that allows users to manage their personal tasks efficiently. Users can **register, log in, create, edit, and delete tasks**, as well as mark them as completed. The application incorporates **secure authentication, session management, and user-specific task tracking**, while providing a **clean, user-friendly interface**. Additionally, it logs user activity for auditing and debugging purposes. This project demonstrates the core concepts of modern web development using Django.

MVT Architecture of the Project

Django follows the **Model-View-Template (MVT)** architecture, which separates data, business logic, and presentation for better maintainability:

Layer	Project Component	Function
Model	`tasks/models.py`	Defines the database structure (Task, User) and handles ORM-based CRUD operations.
View	`tasks/views.py`	Processes user requests, applies business logic, interacts with models, and returns responses.
Template	`tasks/templates/`	Renders dynamic HTML pages, displaying user-specific data in a consistent layout.
Middleware	`tasks/middleware.py`	Intercepts requests and responses for logging, security checks, and session management.
URL Dispatcher	`tasks/urls.py`	Maps URLs to corresponding views, enabling navigation and dynamic routing.

Flow of a Request:

User → Browser → URL → Django URL Dispatcher → View → Model → Template → Response → User

This architecture ensures **clean separation of concerns**, simplifies maintenance, and allows scalability for future enhancements.

Code

I've provided essential code for the project below.

Folder Structure Of Project

```
taskmanager_project/
│
├── taskmanager_project/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── wsgi.py
│   └── asgi.py
│
├── tasks/
│   ├── migrations/
│   │   └── __init__.py
│   ├── __init__.py
│   ├── admin.py
│   ├── apps.py
│   ├── forms.py
│   ├── middleware.py
│   ├── models.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
│
├── templates/
│   └── tasks/
│   ├── base.html
│   ├── login.html
│   ├── register.html
│   ├── task_list.html
│   └── add_task.html
│
├── static/
│   ├── css/
│   └── js/
│
├── manage.py
├── README.md
├── db.sqlite3
└── venv/
```

taskmanager_project/urls.py

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('tasks.urls')),
]
```

tasks/urls.py

```
from django.urls import path
from . import views

urlpatterns = [
    path("register/", views.register, name="register"),
    path("login/", views.user_login, name="login"),
    path("logout/", views.user_logout, name="logout"),
    path("", views.task_list, name="task_list"),
    path("add/", views.add_task, name="add_task"),
    path("edit/<int:id>/", views.edit_task, name="edit_task"),
    path("delete/<int:id>/", views.delete_task, name="delete_task"),
]
```

tasks/models.py

```
from django.db import models
from django.contrib.auth.models import User

class Task(models.Model):
    user = models.ForeignKey(User, on_delete=models.CASCADE)
    title = models.CharField(max_length=200)
    description = models.TextField()
    completed = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.title
```

tasks/forms.py

```
from django import forms
from django.contrib.auth.models import User
from .models import Task
```

```
class TaskForm(forms.ModelForm):
    class Meta:
        model = Task
        fields = ['title', 'description', 'completed']
        widgets = {
            'title': forms.TextInput(attrs={'class': 'form-control'}),
            'description': forms.Textarea(attrs={'class': 'form-control'}),
            'completed': forms.CheckboxInput(attrs={'class': 'form-check-input'})
        }

class RegisterForm(forms.ModelForm):
    password = forms.CharField(widget=forms.PasswordInput(attrs={'class':'form-control'}))
    class Meta:
        model = User
        fields = ['username', 'password']
```

tasks/views.py

```
from django.shortcuts import render, redirect, get_object_or_404
from django.contrib.auth import authenticate, login, logout
from django.contrib.auth.decorators import login_required
from .forms import TaskForm, RegisterForm
from .models import Task
from django.contrib import messages

def register(request):
    if request.method == "POST":
        form = RegisterForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)
            messages.success(request, "Registration successful!")
            return redirect("task_list")
        else:
            messages.error(request, "Invalid data!")
    else:
        form = RegisterForm()
    return render(request, "tasks/register.html", {"form": form})

def user_login(request):
    if request.method == "POST":
        username = request.POST.get("username")
        password = request.POST.get("password")
        user = authenticate(request, username=username, password=password)
        if user:
            login(request, user)
```

```
            messages.success(request, "Login successful!")
            return redirect("task_list")
        else:
            messages.error(request, "Invalid credentials")
    return render(request, "tasks/login.html")

@login_required
def user_logout(request):
    logout(request)
    messages.info(request, "Logged out successfully")
    return redirect("login")

@login_required
def task_list(request):
    tasks = Task.objects.filter(user=request.user)
    return render(request, "tasks/task_list.html", {"tasks": tasks})

@login_required
def add_task(request):
    if request.method == "POST":
        form = TaskForm(request.POST)
        if form.is_valid():
            task = form.save(commit=False)
            task.user = request.user
            task.save()
            messages.success(request, "Task added successfully")
            return redirect("task_list")
    else:
        form = TaskForm()
    return render(request, "tasks/add_task.html", {"form": form})

@login_required
def edit_task(request, id):
    task = get_object_or_404(Task, id=id, user=request.user)
    if request.method == "POST":
        form = TaskForm(request.POST, instance=task)
        if form.is_valid():
            form.save()
            messages.success(request, "Task updated successfully")
            return redirect("task_list")
    else:
        form = TaskForm(instance=task)
    return render(request, "tasks/add_task.html", {"form": form})

@login_required
def delete_task(request, id):
```

```
    task = get_object_or_404(Task, id=id, user=request.user)
    task.delete()
    messages.success(request, "Task deleted successfully")
    return redirect("task_list")
```

tasks/templates/tasks_list.html

```
{% extends "tasks/base.html" %}

{% block content %}
<style>
    :root {
        --primary-gradient: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    }

    .task-container {
        background: #fff;
        border-radius: 15px;
        box-shadow: 0 10px 30px rgba(0, 0, 0, 0.1);
        padding: 2rem;
        margin-top: 2rem;
    }

    .header-section {
        display: flex;
        justify-content: space-between;
        align-items: center;
        border-bottom: 2px solid #f8f9fa;
        padding-bottom: 1.5rem;
        margin-bottom: 1.5rem;
    }

    .page-title {
        font-weight: 700;
        color: #2d3436;
        margin: 0;
    }

    .table {
        vertical-align: middle;
    }

    .table thead th {
        border-top: none;
```

```
        text-transform: uppercase;
        font-size: 0.85rem;
        letter-spacing: 1px;
        color: #636e72;
        padding: 1rem;
    }

    /* Badge Styling for "Completed" status */
    .status-badge {
        padding: 0.5em 1em;
        border-radius: 50px;
        font-size: 0.8rem;
        font-weight: 600;
    }

    .status-yes { background-color: #d1fae5; color: #065f46; }
    .status-no { background-color: #fee2e2; color: #991b1b; }

    /* Button Styling */
    .btn-custom-add {
        background: var(--primary-gradient);
        border: none;
        color: white;
        padding: 0.6rem 1.5rem;
        border-radius: 8px;
        transition: transform 0.2s;
    }

    .btn-custom-add:hover {
        transform: translateY(-2px);
        color: white;
        box-shadow: 0 5px 15px rgba(118, 75, 162, 0.4);
    }

    .action-link {
        text-decoration: none;
        font-weight: 500;
        margin-right: 10px;
        transition: color 0.2s;
    }
</style>

<div class="container">
    <div class="task-container">
        <div class="header-section">
            <h2 class="page-title">My Tasks</h2>
```

```
        <div>
            <a href="{% url 'add_task' %}" class="btn btn-custom-add me-2">
                <i class="bi bi-plus-lg"></i> Add Task
            </a>
            <a href="{% url 'logout' %}" class="btn btn-outline-secondary btn-sm">Logout</a>
        </div>
    </div>

    {% if tasks %}
    <div class="table-responsive">
        <table class="table table-hover">
            <thead>
                <tr>
                    <th>Task Name</th>
                    <th>Description</th>
                    <th class="text-center">Status</th>
                    <th class="text-end">Actions</th>
                </tr>
            </thead>
            <tbody>
                {% for task in tasks %}
                <tr>
                    <td class="fw-bold text-dark">{{ task.title }}</td>
                    <td class="text-muted">{{ task.description|truncatechars:50 }}</td>
                    <td class="text-center">
                        {% if task.completed %}
                            <span class="status-badge status-yes">Completed</span>
                        {% else %}
                            <span class="status-badge status-no">Pending</span>
                        {% endif %}
                    </td>
                    <td class="text-end">
                        <a href="{% url 'edit_task' task.id %}" class="action-link text-
warning">Edit</a>
                        <a href="{% url 'delete_task' task.id %}" class="action-link text-danger"
onclick="return confirm('Delete this task?')">Delete</a>
                    </td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
    {% else %}
    <div class="text-center py-5">
        <img src="https://cdn-icons-png.flaticon.com/512/4076/4076549.png" width="80"
class="mb-3 opacity-50" alt="Empty">
```

```
            <p class="text-muted">Your to-do list is empty. Time to start something new!</p>
            <a href="{% url 'add_task' %}" class="btn btn-primary">Create Your First Task</a>
        </div>
        {% endif %}
    </div>
</div>
{% endblock %}
```

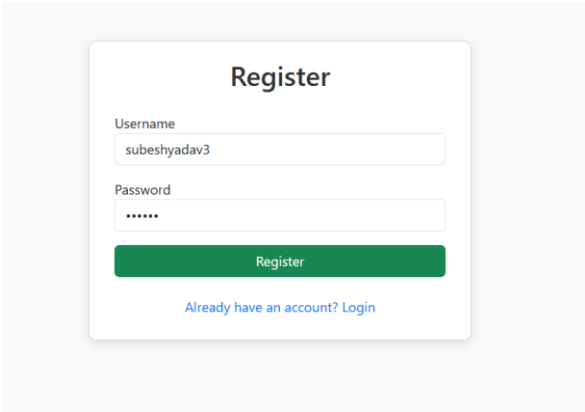
tasks/templates/task_add.html

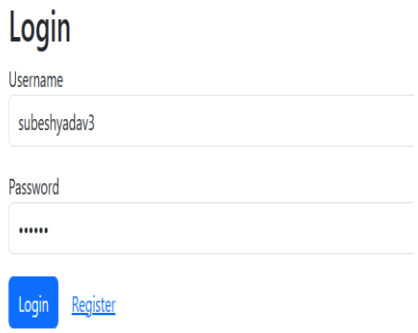
```
{% extends "tasks/base.html" %}

{% block content %}
<style>
    .form-wrapper {
        display: flex;
        justify-content: center;
        align-items: center;
        min-height: 80vh;
    }

    .form-card {
        background: #ffffff;
        border: none;
        border-radius: 15px;
        box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
        padding: 2.5rem;
        width: 100%;
        max-width: 500px;
    }

    .form-title {
        font-weight: 700;
        color: #2d3436;
        margin-bottom: 1.5rem;
        text-align: center;
    }

    .form-label {
        font-weight: 600;
        color: #636e72;
        margin-bottom: 0.5rem;
    }

    .form-control {
        border-radius: 8px;
```

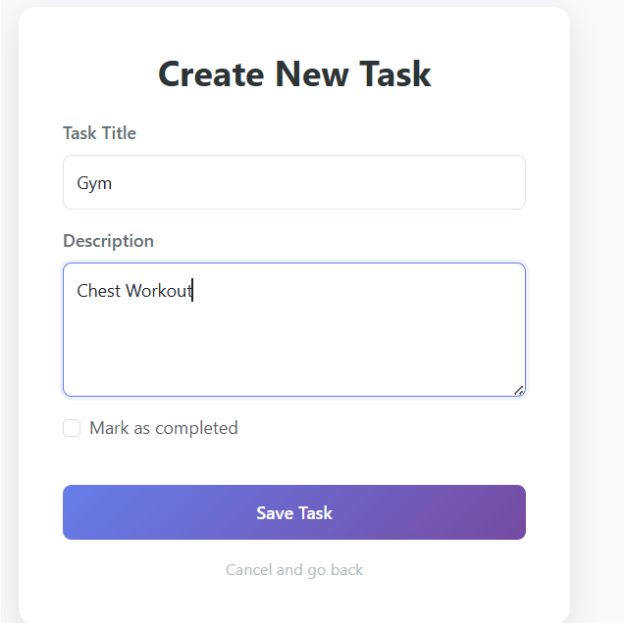
```
    border: 1px solid #dfe6e9;
    padding: 0.75rem;
    transition: all 0.2s;
}

.form-control:focus {
    border-color: #667eea;
    box-shadow: 0 0 0 3px rgba(102, 126, 234, 0.1);
}

.btn-submit {
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    border: none;
    color: white;
    padding: 0.8rem;
    font-weight: 600;
    border-radius: 8px;
    margin-top: 1rem;
    transition: transform 0.2s;
}

.btn-submit:hover {
    transform: translateY(-2px);
    box-shadow: 0 5px 15px rgba(118, 75, 162, 0.3);
}

.btn-cancel {
    background-color: transparent;
    color: #b2bec3;
    border: none;
    text-decoration: none;
    display: block;
    text-align: center;
    margin-top: 1rem;
    font-size: 0.9rem;
}

.btn-cancel:hover {
    color: #636e72;
}


.form-check-input:checked {
    background-color: #667eea;
    border-color: #667eea;
}
```

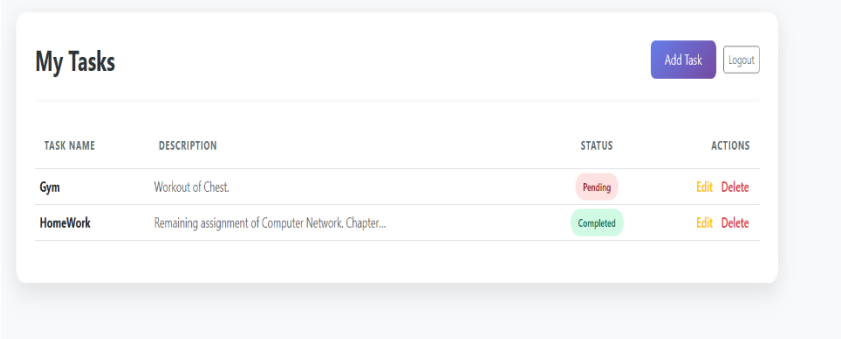
```
</style>

<div class="container form-wrapper">
   <div class="form-card">
      <h2 class="form-title">
         {% if form.instance.id %}Edit Task{% else %}Create New Task{% endif %}
      </h2>

      <form method="POST">
         {% csrf_token %}

         <div class="mb-3">
            <label class="form-label">Task Title</label>
            <input type="text" name="title" value="{{ form.title.value|default:" }}"
                  class="form-control" placeholder="What needs to be done?" required>
         </div>

         <div class="mb-3">
            <label class="form-label">Description</label>
            <textarea name="description" class="form-control" rows="4"
                     placeholder="Add some details..." required>{{ form.description.value|default:"
}}</textarea>
         </div>

         <div class="form-check mb-4">
            <input type="checkbox" name="completed" class="form-check-input"
id="completedCheck"
                  {% if form.completed.value %}checked{% endif %}>
            <label class="form-check-label text-muted" for="completedCheck">
               Mark as completed
            </label>
         </div>

         <button type="submit" class="btn btn-submit w-100">
            {% if form.instance.id %}Update Task{% else %}Save Task{% endif %}
         </button>

         <a href="{% url 'task_list' %}" class="btn-cancel">Cancel and go back</a>
      </form>
   </div>
</div>
{% endblock %}
```

Output










```
(venv) PS F:\Study\5th sem\python\django\taskmanager_project> python manage.py runserver
For more information on production servers see: https://docs.djangoproject.com/en/6.0/howto/deployment/
[2026-03-05 20:14:44] User: subeshyadav3 | SessionID: h20aodglcxuy22ektdbpvpcngs4hqyje | Method: GET | Path: / | Cookies: {'csrftoken': '
o4S9vFRJXb6rxA3g8q0B6JKRw', 'sessionid': 'h20aodglcxuy22ektdbpvpcngs4hqyje'}
[05/Mar/2026 20:14:44] "GET / HTTP/1.1" 200 5216
[2026-03-05 20:15:06] User: subeshyadav3 | SessionID: h20aodglcxuy22ektdbpvpcngs4hqyje | Method: GET | Path: /add/ | Cookies: {'csrftoken
uu8Jo4S9vFRJXb6rxA3g8q0B6JKRw', 'sessionid': 'h20aodglcxuy22ektdbpvpcngs4hqyje'}
[05/Mar/2026 20:15:06] "GET /add/ HTTP/1.1" 200 4156
[2026-03-05 20:15:38] User: subeshyadav3 | SessionID: h20aodglcxuy22ektdbpvpcngs4hqyje | Method: POST | Path: /add/ | Cookies: {'csrftoke
nuu8Jo4S9vFRJXb6rxA3g8q0B6JKRw', 'sessionid': 'h20aodglcxuy22ektdbpvpcngs4hqyje'}
[05/Mar/2026 20:15:38] "POST /add/ HTTP/1.1" 302 0
[2026-03-05 20:15:38] User: subeshyadav3 | SessionID: h20aodglcxuy22ektdbpvpcngs4hqyje | Method: GET | Path: / | Cookies: {'csrftoken': '
o4S9vFRJXb6rxA3g8q0B6JKRw', 'sessionid': 'h20aodglcxuy22ektdbpvpcngs4hqyje', 'messages': 'W1siX19qc29uX21lc3NhZ2UiLDAsMjUsIlRhc2sgYWRkZWQ
Vzc2Z1bGx5IiwiIl1d:1vy9io:AK_9-ZUyvOueqSeLUvNavXiv0M8qlAzHpQWO_RcEYYY'}
[05/Mar/2026 20:15:38] "GET / HTTP/1.1" 200 6051
```

Discussion and Conclusion

The **Student Task Manager** project demonstrates how Django can be used to build a fully functional web application with secure user authentication, session management, and task management features. The project shows practical implementation of core concepts such as handling requests and responses, form data validation, routing, templating, CRUD operations with ORM, and middleware for logging and security. By creating, editing, and deleting tasks, users experience a responsive and interactive system while the backend efficiently handles data and maintains user-specific states.

This project also highlights the importance of **structured architecture (MVT)**, separation of concerns, and secure coding practices in web development. Through this application, concepts like cookies, sessions, and authentication are applied practically, and logging middleware provides insight into user actions for monitoring and debugging. Overall, it reinforces a strong understanding of Django's capabilities, demonstrating both front-end usability and robust backend functionality in a cohesive, maintainable system.